

Algorithmes et Pensée Computationnelle

Algorithmes probabilistes

Le but de cette séance est de comprendre les algorithmes probabilistes. Ceux-ci permettent de résoudre des problèmes complexes en relativement peu de temps. La contrepartie est que le résultat obtenu est généralement une solution approximative du problème initial. Néanmoins, ces algorithmes demeurent très utiles pour beaucoup d'applications.

1 Monte-Carlo

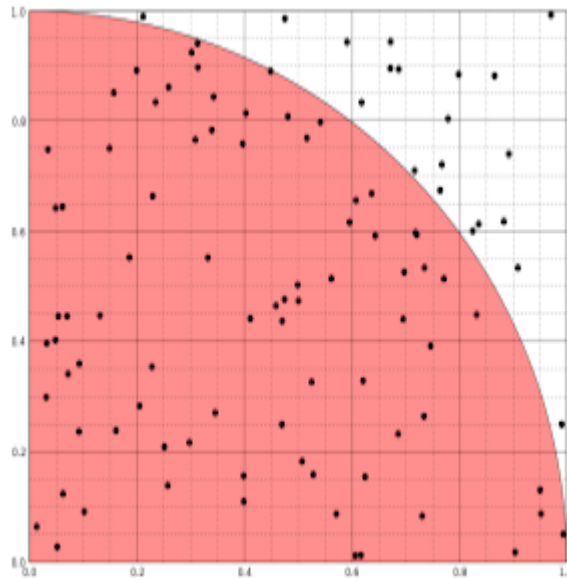
Question 1: (🕒 10 minutes) Un jeu de hasard : Python

Supposez que vous lanciez une pièce de monnaie k fois et que vous voulez calculer la probabilité d'avoir un certain nombre de piles. Vous devez programmer un algorithme probabiliste, permettant de calculer cette probabilité. Pour ce faire, vous devez compléter la fonction `proba(n, k, iter)` contenue dans le fichier `piece.py` dans le dossier `code`. Cette fonction renvoie la probabilité approximative d'avoir n pile en k lancers de pièce.

La fonction `piece(k)` permet de créer une liste de taille k contenant des 0 et des 1 aléatoirement avec une probabilité $\frac{1}{2}$. Considérez un chiffre 1 comme une réussite (pile) et 0 comme un échec (face).

Question 2: (🕒 15 minutes) Une approximation de π : Python

L'objectif de cet exercice est d'écrire un algorithme probabiliste permettant d'approximer le nombre π . Imaginez un plan sur lequel $0 \leq x \leq 1$ et $0 \leq y \leq 1$. Sur ce dernier, nous allons dessiner un quart de cercle centré en $(0,0)$ et avec un rayon de 1. Par conséquent, un point dans cet espace se trouve à l'intérieur ou sur le cercle si $x^2 + y^2 \leq 1$. Vous trouverez ci-dessous une illustration de la situation :



La première étape de cet exercice consiste à créer une fonction permettant de déterminer si un point est à l'intérieur (zone rouge) ou à l'extérieur du cercle. Puis, générez 10000 points dans cet espace (x et y devraient appartenir à l'intervalle $[0,1]$). Pour ce faire, vous pouvez utiliser la fonction `random.random()` après avoir importé le module `random`. Vous pouvez obtenir l'approximation de π à partir de la formule suivante : $\pi \approx \left(\frac{\text{Nombre de points dans le quart de cercle}}{\text{Nombre total de points}} \right) \cdot 4$. Votre réponse devrait être assez proche de la vraie valeur de π .

2 Fingerprinting

Question 3: (🕒 15 minutes) Fingerprinting : Une mission pour l'agente secrète Alice : Python

Dans cet exercice, vous prendrez le rôle de l'agente secrète Alice. Cette dernière enquêtait sur la disparition de son collègue, l'agent Bob, et se doutait que l'indice clé qui la mènerait à la vérité se trouvait dans la boîte mail de Bob. Alice arriva à trouver un bout de papier avec écrit dessus : *"Mon mot de passe est l'empreinte de ceci est mon mot de passe"*. Aidez Alice à trouver l'empreinte du mot de passe !

Pour cela, vous devez compléter deux fonctions :

1. `is_a_prime_number(num)` qui vérifie que `num` est un nombre premier ou pas. Un nombre premier est un entier naturel qui admet exactement deux diviseurs distincts entiers et positifs. Ces deux diviseurs sont 1 et le nombre considéré, puisque tout nombre a pour diviseurs 1 et lui-même, les nombres premiers étant ceux qui n'en possèdent aucun autre.
2. `fingerprinting(p, message)` qui implémente l'algorithme de fingerprinting suivant :
 - (a) Si `p` est un nombre premier, calculez la valeur de hachage de la chaîne à l'aide de la fonction `hash(...)`, puis calculez le modulo du résultat du hachage avec `p`.
 - (b) Sinon, affichez un message qui dit que le nombre n'est pas un nombre premier.

Si vous réussissez à implémenter les deux fonctions correctement, le programme vous affichera : Connexion réussie? `True`.

À vos ordres, détectives !

```
1 import base64
2
3 # num est un nombre entier
4 def is_a_prime_number(num):
5     # Partie à compléter
6
7
8 # p est un nombre premier et message est une chaîne de caractères
9 def fingerprinting(p, message):
10    # Partie à compléter
11
12
13 # password est une chaîne de caractères et your_details est un tuple avec le
14 # format suivant (nombre premier, hash du mot de passe)
15 def login(password, your_details):
16     return your_details[1] % your_details[0] ==
17         fingerprinting(your_details[0], password)
18
19
20 # Début de votre programme (ne pas modifier)
21 if __name__ == '__main__':
22     password = "ceciestmonmotdepasse"
23     your_details = (19, hash(password))
24     success = login(password, your_details)
25
26     print("Connexion réussie? " + str(success))
27     if success:
28         message = '''SmUgc2VyYWlzlGNvbmlZpbnOpIGNoZXogbWVzIHBhcmVudHMgw
29                     6AgbGEgY2FtcGFnbmUgbGVzIGRldXggcHJvY2hhaW5lIHNlbWF
30                     pbmVzLCBl dCBqZSBuJ2F1cmFpcyBwYXMGYWNjw6hzIMogIEludGV
31                     ybmV0LiDDgCBiaWVudMO0dCE='''
32     print(base64.b64decode(message).decode())
```

3 Las Vegas

Question 4: (🕒 10 minutes) Un point dans un cercle unitaire : Python Optionnel

Les algorithmes de Monte Carlo sont des algorithmes probabilistes dont la sortie peut être incorrecte avec

une certaine probabilité, qui est généralement faible. En revanche, un algorithme de Las Vegas est un algorithme probabiliste qui trouve toujours le bon résultat lorsqu'il existe. Son inconvénient est que sa complexité temporelle ne peut être garantie à l'avance car elle dépend des données passées en paramètres.

L'objectif de cet exercice est d'écrire un algorithme probabiliste permettant de générer les coordonnées d'un point contenu dans le même quadrant d'un cercle unitaire que pour l'exercice 2. Pour cela, vous pouvez générer des points jusqu'à ce que vous trouviez un point qui soit contenu dans le cercle.

4 Treap

Question 5: (🕒 30 minutes) Insertion dans un Treap : Python et Papier

Un arbre binaire de recherche montre de meilleures performances lorsqu'il est équilibré. Ainsi, la complexité d'une opération de recherche, d'insertion ou de suppression d'un nœud dans l'arbre équilibré a est de $O(\log n)$. Cette complexité est de $O(n)$ dans l'arbre b .

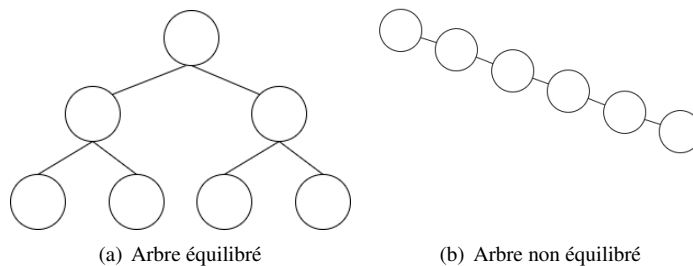


FIGURE 1 – Exemple d'arbres binaires de recherche

Afin de s'assurer d'obtenir un arbre binaire de recherche presque équilibré, on peut utiliser les propriétés d'un heap (ou tas) dont les éléments sont ordonnés en suivant une priorité. Dans un Max-heap (Figure 2), le nœud ayant la priorité maximale se trouve toujours au sommet de l'arbre. Les nœuds parents auront toujours une priorité plus grande que les nœuds enfants. Dans un Min-heap tel que présenté en cours, le nœud ayant la plus faible priorité se trouve au sommet et dans cette configuration, les nœuds parents auront toujours une priorité plus petite que les nœuds enfants.

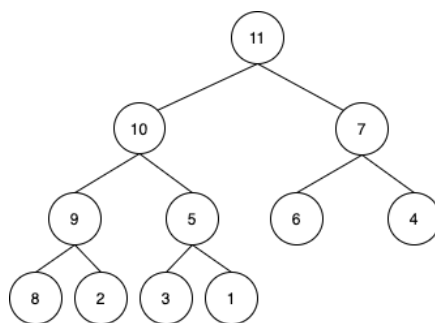


FIGURE 2 – Exemple de Max-heap

Un treap (ou arbretas) est une fusion entre un arbre binaire de recherche et un Heap. En construisant un treap, vous devez vous assurer que les deux conditions ci-dessous soient respectées :

- Le nœud de gauche a une valeur plus petite que le nœud parent, tandis que le nœud de droite a toujours une valeur plus grande que le nœud parent. *—propriété d'un arbre binaire de recherche*
- Chaque enfant a une priorité plus petite que la priorité du parent. *—propriété d'un max-heap*

Soit la liste suivante : `liste = [5, 2, 1, 4, 9, 8, 10]`. Dans le fichier `code/treap.py` se trouvant sur Moodle, créer une nouvelle liste `nodes` qui contient des ensembles de tuples à deux éléments. Chaque tuple contiendra un élément de `liste` représentant une clé et une valeur aléatoire r (comprise entre 0 et 99) représentant une priorité.

 Attention

Notez ci-dessous les valeurs aléatoires ($r_0, \dots, r_6$) que vous obtiendrez :

`nodes = [(5,  $r_0$ ), (2,  $r_1$ ), (1,  $r_2$ ), (4,  $r_3$ ), (9,  $r_4$ ), (8,  $r_5$ ), (10,  $r_6$ )]`

Placez de manière itérative les éléments de `nodes` dans un Treap. Utilisez l'espace ci-dessous pour dessiner le Treap qui sera obtenu.

 Figure